Desenvolvimento de Quiz em C: Uma Abordagem Dinâmica com Estruturas Encadeadas e Recursividade

Bruno Salvador
Paulo Lucas
Professor: Aldo Henrique
Centro universitário Unieuro
bruno090507@unieuro.com.br
paulo093166@unieuro.com.br

I. INTRODUÇÃO

A linguagem C é amplamente reconhecida por seu alto desempenho e pelo controle granular que oferece ao programador sobre os recursos do sistema [1]. Em disciplinas de Estrutura de Dados, compreender como agrupar informações heterogêneas, manipular seus endereços de memória e gerenciar o fluxo de execução é fundamental para o desenvolvimento de softwares eficientes e otimizados [2].

Neste contexto, o presente projeto tem como objetivo desenvolver um jogo interativo do tipo "Quiz" no terminal, focado em testar conhecimentos sobre a própria linguagem C. O programa visa transcender a programação estruturada básica, aplicando na prática conceitos avançados de Estruturas de Dados Dinâmicas, rompendo com o uso de alocações estáticas e laços de repetição convencionais.

Para alcançar este objetivo, o sistema foi projetado para abrigar um banco de 30 questões estruturado em memória dinâmica. A cada rodada, o programa embaralha este banco e seleciona uma amostra (até 10 perguntas), garantindo uma experiência imprevisível ao usuário, e culmina em um sistema de Ranking. O desenvolvimento exigiu a implementação de Listas Encadeadas, Pilhas, algoritmos de ordenação (Selection Sort), chamadas recursivas e tratamentos rigorosos no fluxo de entrada de dados do teclado.

II. METODOLOGIA

A arquitetura do código foi construída e refatorada sob quatro pilares principais:

1. Estruturas de Dados Dinâmicas (Listas e Pilhas): Os dados foram encapsulados via `typedef struct` em duas arquiteturas distintas. A primeira, `NoPergunta`, atua como uma Lista Encadeada (Linked List), onde cada nó guarda o enunciado, alternativas, gabarito e um ponteiro para a próxima questão, facilitando a escalabilidade do banco. A segunda, `NoPilha`, implementa uma Pilha (Stack) para o histórico de jogadores, onde o último registro de pontuação é inserido no topo da estrutura.

2. Alocação e Liberação Dinâmica (Malloc e Free): O sistema abandona vetores globais e gerencia o hardware sob demanda. A inserção de cada nova pergunta no banco ou de um novo jogador no placar invoca a função `malloc` e o operador `sizeof()` para reservar memória exata em tempo de execução. Ao final do programa, rotinas de varredura utilizam a instrução `free` para destruir todos os nós da lista e da pilha, devolvendo a memória ao Sistema Operacional e prevenindo vazamentos (*memory leaks*).

3. Recursividade e Algoritmos de Ordenação: O fluxo da partida substitui laços iterativos (`while`/`for`) por Recursividade. A função `jogarQuizRecursivo` exibe a questão, avalia a resposta e invoca a si mesma passando o ponteiro do próximo nó, até atingir a condição de parada. Para a exibição do Ranking, aplicou-se o algoritmo *Selection Sort* sobre um vetor alocado dinamicamente, extraindo os dados da Pilha e ordenando os jogadores de forma decrescente por pontuação.

4. Entrada de Dados Segura (Sanitização de Buffer): A interação humana foi blindada contra falhas clássicas de leitura. Substituiu-se o uso do `scanf` puro pela combinação de `fgets` (que consome a linha inteira, incluindo quebras de linha `\n`) e `sscanf` (para extração segura do tipo de dado desejado). A validação de caracteres (maiúsculas e minúsculas) foi otimizada deduzindo 32 do valor decimal da Tabela ASCII inserido pelo usuário.

Isso facilitou enormemente a manipulação do banco de 30 questões originais.

III. RESULTADOS

Figura 1: Tela inicial do programa e exibição da primeira pergunta embaralhada.

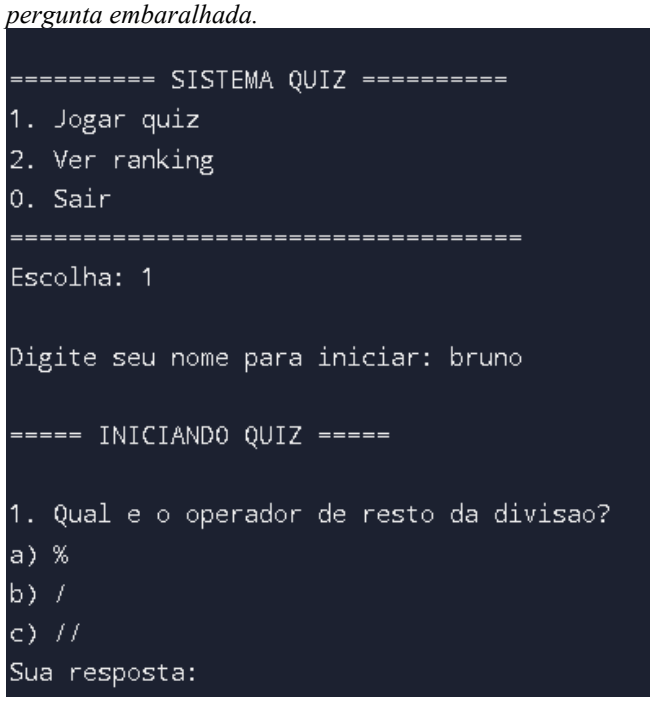


Figura 2: Entrada de dados blindada via *fgets/sscanf* e feedback imediato de validação de caracteres via tabela ASCII..

```c
fgets(linha, sizeof(linha), stdin);
sscanf(linha, " %c", &resp);

if (resp == no->resposta_correta || resp == (no->resposta_correta - 32)) {
    printf("-> Acertou!\n");
    (*acertos)++;
} else {
    printf("-> Errou! Era a letra '%c'.\n", no->resposta_correta);
}
```

Figura 3: Saída final exibindo o Ranking Geral ordenado pelo algoritmo *Selection Sort* e encerramento com limpeza total de memória

```
========== SISTEMA QUIZ ==========
1. Jogar quiz
2. Ver ranking
0. Sair
==================================
Escolha: 2

============ RANKING GERAL ============
1º lugar: paulo           | Pontos: 70
2º lugar: Bruno           | Pontos: 40
=======================================
```

IV. DISCUSSÃO

A implementação deste quiz evidenciou que o domínio de Estruturas de Dados e do controle manual de memória oferece uma eficiência arquitetônica muito superior a métodos puramente estáticos. O uso de Listas Encadeadas garantiu a flexibilidade do banco de questões, permitindo um embaralhamento seguro que apenas permuta os dados internos dos nós, preservando a integridade dos ponteiros.

O avanço central do projeto, no entanto, concentrou-se na união da alocação dinâmica com lógicas avançadas, como a Recursão, que provou ser uma alternativa elegante e eficiente para o controle de estados do jogo. O uso da Pilha para o histórico de pontuações, somado ao algoritmo *Selection Sort* em vetores temporários, demonstrou capacidade de manipular e ordenar conjuntos de dados complexos sob demanda. Além disso, a substituição da limpeza manual de buffer por funções robustas (`fgets`) mitigou o comportamento imprevisível do teclado, prática essencial para um software profissional.

Embora o controle manual exija extremo rigor — garantido pelas rotinas de `free` executadas na `main` —, a aplicação conjunta dessas técnicas resultou em um sistema altamente estável e escalável.

V. CONCLUSÃO

A elaboração deste projeto demonstrou, de forma prática e metódica, a importância do gerenciamento consciente da memória e do domínio arquitetônico em C. Compreendeu-se que o uso de Listas Encadeadas e Pilhas torna o software dinâmico e inteligente na gestão de recursos do hardware. A substituição de laços comuns por Recursividade e a aplicação de algoritmos de ordenação provaram que a linguagem oferece ferramentas poderosas para a resolução de problemas lógicos complexos. Por fim, as tratativas rigorosas de entrada de dados reforçam o princípio de que o desenvolvimento de software deve sempre focar na previsibilidade, na segurança da execução e na tolerância a falhas na interação humana.

REFERÊNCIAS

[1] KERNIGHAN, B. W.; RITCHIE, D. M. *C: a linguagem de programação*. 2. ed. Rio de Janeiro: Campus, 1989.

[2] SCHILDT, H. *C completo e total*. 3. ed. São Paulo: Makron Books, 1997.

[3] Material didático e de laboratório da disciplina de Algoritmos e Estrutura de Dados, Centro Universitário Unieuro, 2026.